

T1: Problemas Clássicos de Concorrência

FPPD - Fundamentos de Processamento Paralelo e Distribuído (98713-04) | **Eduardo Hoffmann e Lucas Mocelin**

Repositório: <https://github.com/theeduardohoffmann/go-concurrency-benchmarks>

Ambiente: Go 1.27, Windows 11, 12 CPUs lógicas. Os dois programas foram executados com `go run -race` sem nenhum aviso de data race; os dados abaixo foram coletados sem `-race`, que distorce o tempo. O `time.Sleep` do Windows tem granularidade de cerca de 1 ms, então os tempos servem para comparar configurações entre si.

1. Problema 1: Jantar dos Filósofos

1.1 Solução e decisões de projeto

Cada garfo é um `chan struct{}` de capacidade 1: um token no channel significa garfo livre; pegar é receber e soltar é enviar de volta. Cada filósofo é uma goroutine que repete pensar, pegar os garfos, comer e soltar. O programa recebe `-n` (N) e `-r` (R). Interpretamos R como o **número total de refeições da mesa** (um contador atômico); cada filósofo come o quanto conseguir, e a distribuição das refeições mostra se a estratégia é justa. Se cada filósofo fizesse R refeições, todos terminariam com o mesmo número por construção e não haveria o que comparar.

Versão base: pega o garfo da esquerda, espera 2 ms e pega o da direita; a pausa torna o deadlock reproduzível (ela vale para todas as versões, para a comparação ser justa). **Estratégias que eliminam o deadlock:** (1) **hierarquia de recursos:** os garfos são numerados e cada filósofo pega primeiro o de menor número (só o filósofo N-1 inverte a ordem); (2) **garçom:** uma goroutine recebe os pedidos por um channel em ordem de chegada, retira os dois garfos do filósofo e só então responde; o filósofo come e devolve os garfos.

1.2 Encerramento ordenado

Um filósofo termina quando as R refeições acabam; a main espera todos com `sync.WaitGroup`. No garçom, em seguida a main fecha o channel de pedidos, o `for` range do garçom termina e a main espera o aviso de fim. A espera por um garfo é um `select` com `time.After` de 1 s: se um filósofo espera tanto, o programa conclui que há deadlock, imprime quem espera qual garfo e a goroutine retorna. Assim, até a versão que trava termina sem goroutines bloqueadas.

1.3 Análise de deadlock e starvation

Condições de Coffman: (a) exclusão mútua, (b) posse e espera, (c) não preempção, (d) espera circular. O deadlock exige as quatro; cada estratégia quebra uma.

Versão	Deadlock?	Justificativa (Coffman)	Starvation?
base	Sim , nas 3 execuções. Todos ficam segurando o garfo <code>i</code> e esperando o <code>i+1</code> (0 espera 1, 1 espera 2, ..., 4 espera 0).	As quatro condições valem: o garfo é exclusivo, cada um segura um e pede outro, ninguém é forçado a soltar e as esperas formam um ciclo.	Sim, como consequência: ninguém mais come.
hierarquia	Não (3 execuções).	Quebra a espera circular (d): todos pegam em ordem crescente, então um ciclo de espera exigiria uma ordem decrescente em algum ponto, o que não existe.	Não há starvation total, mas é injusta : os filósofos 0 e 4 comem bem menos (ver 1.4).
garçom	Não (3 execuções).	Quebra a posse e espera (b): só o garçom retira garfos e o filósofo só recebe quando já tem os dois. Quem segura garfos está comendo e vai soltá-los.	Não: o garçom atende em ordem de chegada (fila FIFO), então ninguém é ultrapassado.

A ausência de starvation nas versões corretas depende de a refeição durar um tempo finito e de as filas de espera dos channels do Go serem servidas em ordem de chegada (comportamento do runtime). As condições (a) e (c) não são eliminadas em nenhuma versão, pois são inerentes a garfos físicos.

1.4 Dados de execução e fairness

Execução	Versão	Refeições por filósofo (F0 F1 F2 F3 F4)	Espera média pelos garfos, ms (F0 ... F4)
1	base	127 128 128 128 128 (deadlock; total 639)	4,30 4,11 4,14 4,39 4,20
2	base	0 0 0 0 0 (deadlock)	-
3	base	0 0 0 0 0 (deadlock)	-
1	hierarquia	161 202 215 260 162	4,71 3,15 2,73 1,89 4,66
2	hierarquia	161 201 217 261 160	4,64 3,27 2,76 1,88 4,89
3	hierarquia	159 205 221 255 160	4,72 3,13 2,67 1,90 4,83
1	garçom	203 195 203 198 201	1,81 1,85 1,85 1,85 1,82
2	garçom	204 200 199 197 200	1,86 1,95 1,86 2,08 1,92
3	garçom	204 202 201 195 198	1,81 1,83 1,91 1,91 1,97

N=5, R=1000 refeições no total, pensar e comer até 2 ms. Nas execuções com deadlock, as refeições são as feitas antes de travar.

Hierarquia evita o deadlock, mas é injusta: os filósofos 0 e 4 comem cerca de 160 vezes e esperam ~4,7 ms por refeição, enquanto o filósofo 3 come cerca de 260 vezes e espera ~1,9 ms (2,5 vezes menos). O padrão se repetiu nas 3 execuções, então não é acaso. Nossa hipótese (não testada separadamente) é que os filósofos 0 e 4 pegam ambos primeiro o garfo 0 e o seguram enquanto esperam o segundo garfo, atrapalhando-se, enquanto o filósofo 3 tem menos concorrência. O **garçom** é justo: todos comem entre 195 e 204 vezes e esperam de 1,8 a 2,1 ms. A fila FIFO do garçom é o que garante isso, e a hierarquia, que só ordena os recursos, não. Na versão base, as refeições de antes do deadlock são iguais para todos, mas não dizem nada sobre fairness, pois o sistema trava.

2. Problema 2: Produtor/Consumidor com buffer limitado

2.1 Solução e decisões de projeto

As duas versões implementam a mesma interface (put, get, getTimeout, tamanho), então produtores e consumidores são o mesmo código nas duas. **Versão com channel:** o buffer é `make(chan int, K)`; o próprio Go bloqueia o produtor com o buffer cheio e o consumidor com o buffer vazio. **Versão com semáforos:** o buffer é um slice circular de K posições, com dois semáforos de contagem, `notFull` (começa em K) e `notEmpty` (começa em 0), e dois mutexes. O semáforo é um channel de tokens (P = receber, V = enviar), pois a biblioteca padrão não tem semáforo; o buffer de itens é o slice. O algoritmo é: put: `P(notFull); Lock(mEntra); itens[entra]=x; entra++; Unlock(mEntra); V(notEmpty)` e get: `P(notEmpty); Lock(mSai); x=itens[sai]; sai++; Unlock(mSai); V(notFull)`. Com vários produtores e consumidores os semáforos sozinhos não bastam: sem `mEntra` dois produtores poderiam receber a mesma posição de escrita e um sobrescreveria o outro; sem `mSai` dois consumidores poderiam retirar o mesmo item. Usamos dois mutexes distintos para que um produtor e um consumidor trabalhem ao mesmo tempo. O semáforo é obtido antes do mutex, para ninguém ficar bloqueado esperando vaga ou item segurando o mutex. Não há data race entre escrita e leitura da mesma posição porque o `V(notEmpty)` só ocorre depois da escrita e o `V(notFull)` depois da leitura, o que cria a relação happens-before do modelo de memória de Go (confirmado com `-race`).

Consumo com timeout: o consumidor 0 usa `select` com `time.After` nas duas versões. Se nenhum item chega em 5 ms (configurável), ele executa o comportamento alternativo (conta o timeout e imprime uma mensagem nas 3 primeiras vezes) e volta a tentar. **Verificação:** cada item tem um identificador único; ao final o programa confere se o total consumido é igual ao produzido e se nenhum identificador foi perdido ou consumido duas vezes. Todas as linhas dos resultados abaixo terminaram com `ok=true` (verificação aprovada).

2.2 Encerramento ordenado

Depois que todos os produtores terminam (`WaitGroup`), a main coloca no buffer uma **pílula de fim** (o valor -1) para cada consumidor. Como o buffer é FIFO, as pílulas ficam atrás de todos os itens reais: um consumidor só recebe a pílula depois que os itens anteriores foram retirados, então o buffer é drenado e nenhum item se perde. Cada consumidor encerra ao receber uma pílula e a main espera todos terminarem. É o mesmo mecanismo nas duas versões, inclusive para o consumidor 0 (com timeout). Não há deadlock porque há exatamente uma pílula por consumidor.

2.3 Deadlock e starvation

Não ocorreu deadlock em nenhuma das versões: todas as execuções terminaram, inclusive com `K=1`, que é o caso de mais bloqueio. Os produtores só esperam vaga, que os consumidores liberam, e os consumidores só esperam itens, que os produtores entregam; não há dependência circular e nenhum mutex é mantido durante uma espera. Starvation: as filas de espera dos channels são servidas em ordem de chegada, então nenhum produtor ou consumidor é ignorado indefinidamente; na carga com trabalho cada consumidor processou entre 1964 e 2028 dos ~2000 itens.

2.4 Efeito do buffer (K) e comparação entre as versões

Fixamos `P=4` e `C=4` e variamos `K` em 1, 10 e 100, em duas cargas: **com trabalho simulado** (cada produtor e consumidor gasta em média 0,5 ms por item), que representa o uso normal e mostra o desacoplamento; e **sem trabalho**, que mede só o custo de sincronização e é onde as duas versões diferem.

Carga	Versão	K	Itens/s (throughput)	Ocupação média (itens)	Itens por consumidor (C0 C1 C2 C3)
Com trabalho (2000 itens por produtor)	channel	1	4459	0,52	2021 1993 1975 2011
	channel	10	4660	5,66	2028 1964 1994 2014
	channel	100	4727	18,27	1979 2012 2003 2006
	semáforos	1	4460	0,51	2009 1997 1992 2002
	semáforos	10	4635	4,05	1989 2021 1994 1996
	semáforos	100	4662	15,66	2009 1998 1985 2008
Sem trabalho (50000 itens por produtor)	channel	1	2.654.224	0,45	25370 57437 55723 61470
	channel	10	5.297.564	5,06	20651 57007 70073 52269
	channel	100	5.451.433	62,94	17235 58750 64119 59896
	semáforos	1	1.444.110	0,00	49490 50281 49864 50365
	semáforos	10	2.003.265	2,56	42248 52111 51580 54061
	semáforos	100	2.637.732	58,69	43901 51188 51483 53428

Uma execução por configuração. Ocupação = média de amostras do tamanho do buffer tiradas a cada 200 µs. Throughput = itens consumidos / tempo total.

Efeito de K e desacoplamento. Com trabalho simulado o throughput quase não muda com K (de ~4460 com K=1 para ~4650-4730 com K=10 e K=100, ganho de ~5%), porque o limite é o tempo de trabalho. O efeito de K aparece na ocupação: com K=1 o buffer tem em média 0,5 item (produtor e consumidor praticamente em sincronia, um esperando o outro), e com K=10 e K=100 a fila de itens cresce para ~4-6 e ~16-18. Ou seja, o buffer maior absorve as variações aleatórias de velocidade entre produtores e consumidores, desacoplando os dois; a maior parte do ganho de vazão já aparece em K=10 e depois não muda. Sem trabalho simulado o efeito é grande: no channel o throughput passa de 2,65 para 5,30 e 5,45 milhões de itens/s (2,1 vezes de K=1 para K=100), e nos semáforos de 1,44 para 2,64 milhões (1,8 vezes). Com buffer maior, produtores e consumidores bloqueiam e acordam com menos frequência, então há menos trocas entre goroutines (explicação coerente com os dados, não medida diretamente).

Channel x semáforos. Com trabalho simulado as duas versões são indistinguíveis (diferenças de ~1%, dentro do ruído). Sem trabalho, o channel foi de 1,8 a 2,6 vezes mais rápido. Uma explicação possível é que o channel faz uma operação por item, que o runtime do Go já implementa com buffer circular e lock interno, enquanto a versão com semáforos faz duas operações de channel (P e V) mais um mutex por item. Os dois caminhos bloqueiam do mesmo modo; a diferença é o custo de sincronização por item. Observação: sem trabalho, o consumidor 0 consumiu bem menos itens no channel (17 a 25 mil contra 52 a 70 mil dos outros). Atribuímos isso (hipótese) ao custo de criar um timer a cada chamada de `time.After`, e não a injustiça do buffer; nos semáforos a diferença é menor (42 a 49 mil contra ~52 mil).

Demonstração do timeout. Com produtores lentos (3 ms por item) e consumidores sem trabalho, o buffer fica vazio e o consumidor 0 estourou o timeout de 2 ms 640 vezes no channel e 622 nos semáforos, executando a tarefa alternativa. Nesse caso ele processou só 139 e 157 itens, contra ~620 dos demais, pois a cada timeout volta ao fim da fila de espera. A verificação continuou aprovada (2000 itens produzidos e consumidos).

3. Metodologia, validação e limitações

Corretude. Os dois programas foram executados com `go run -race` em todas as configurações usadas neste relatório, sem nenhum aviso de data race. No Problema 1, o deadlock da base foi reproduzido em todas as execuções e as duas estratégias terminaram sem deadlock e sem goroutines bloqueadas em todas as execuções. No Problema 2, toda execução terminou com a verificação aprovada: total produzido igual ao consumido e nenhum item perdido ou repetido. Os parâmetros inválidos (N menor que 2, P ou C menores que 1, estratégia desconhecida) são rejeitados com mensagem.

Como reproduzir os dados. Problema 1: três execuções de `go run . -r 1000` (tabela da seção 1.4). Problema 2: `go run . -itens 2000` (carga com trabalho simulado, K = 1, 10 e 100), `go run . -itens 50000 -prod 0s -cons 0s` (sem trabalho) e `go run . -k 10 -itens 500 -prod 3ms -cons 0s -timeout 2ms` (demonstração do timeout). Os valores exatos mudam a cada execução (os tempos são aleatórios e dependem da máquina); o padrão qualitativo se repete.

Limitações. (1) Cada configuração do Problema 2 foi medida uma vez, então diferenças pequenas entre linhas são ruído e não foram interpretadas. (2) A ausência de starvation depende de as filas de espera dos channels do Go serem atendidas em ordem de chegada, o que é comportamento do runtime e não garantia da especificação. (3) No Windows o `time.Sleep` tem granularidade de cerca de 1 ms, então os tempos servem para comparar configurações entre si. (4) As explicações marcadas como hipótese (a injustiça da hierarquia e o menor consumo do consumidor 0 sem trabalho simulado) não foram verificadas por um experimento específico.

4. Uso de ferramentas de IA

Ferramenta utilizada: Claude Code (modelo Claude Sonnet 5.5, da Anthropic), em sessão de conversa com o grupo. O trabalho foi desenvolvido pelos integrantes em colaboração com a ferramenta, em ciclos de pedido, revisão e ajuste. A tabela abaixo detalha, por etapa, o que coube ao grupo e o que coube à ferramenta.

Etapa	Contribuição do grupo	Contribuição da ferramenta
Projeto da solução	Definiu o escopo a partir do enunciado e decidiu reduzi-lo ao que é exigido: por isso a estratégia de limite de N-1 filósofos foi retirada, ficando hierarquia de recursos e garçom. Definiu que a solução deveria ser simples o bastante para ser explicada na apresentação.	Propôs as alternativas de projeto (estratégias do Problema 1, semáforos com dois mutexes, encerramento por pílulas de fim, interpretação de R como total de refeições da mesa) e justificou cada uma.
Geração de código	Conduziu várias rodadas de revisão: pediu a retirada de tudo que não era exigido, a divisão em funções menores, a remoção de comentários e a redução dos valores padrão para a demonstração; criou o repositório Git e fez os commits.	Escreveu o código dos dois programas e as sucessivas versões simplificadas, a partir das instruções do grupo.
Depuração e validação	Executou os programas em suas máquinas com <code>go run -race</code> e conferiu os resultados; autorizou a instalação das ferramentas necessárias (Go e gcc de 64 bits).	Executou os programas, encontrou e corrigiu falhas (validação de parâmetros, estratégia inválida tratada como base) e conferiu a ausência de <code>data race</code> .
Análise dos dados	Revisou as conclusões e estudou o código e as análises para a apresentação.	Coletou as saídas, organizou as tabelas e redigiu as interpretações, marcando como hipótese o que não foi testado.
Redação do relatório	Orientou o conteúdo exigido, o nível de detalhe e os ajustes de texto.	Redigiu o relatório e o README conforme as orientações do grupo.

Todos os números deste relatório vêm de execuções reais dos programas entregues; nenhum foi estimado. Cada integrante do grupo deve ser capaz de explicar qualquer parte do código e das análises, conforme o item 4 das Regras Gerais.

5. Referências

[1] Material da disciplina FPPD (Unidades 1, 2 e 3): condições de Coffman, semáforos e algoritmo clássico do buffer limitado.

[2] DIJKSTRA, E. W. Hierarchical ordering of sequential processes. Acta Informatica, v. 1, n. 2, 1971. (jantar dos filósofos e ordenação hierárquica de recursos)

[3] COFFMAN, E. G.; ELPHICK, M.; SHOSHANI, A. System deadlocks. ACM Computing Surveys, v. 3, n. 2, 1971. (condições necessárias para o deadlock)

[4] The Go Programming Language Specification: channels, select e go statements. <https://go.dev/ref/spec>

[5] The Go Memory Model (relação happens-before entre envio e recebimento em channels). <https://go.dev/ref/mem>

[6] Documentação dos pacotes `sync`, `sync/atomic` e `time` e do detector de `data race`. <https://pkg.go.dev> e https://go.dev/doc/articles/race_detector